

校园中转分发与跑腿服务管理系统——新版10人详细分工方案

一、分工调整背景

本项目为数据库课程设计项目，系统名称为 **校园中转分发与跑腿服务管理系统**。

项目采用 **ASP.NET Core MVC + Oracle 19c + C# + Bootstrap** 技术路线，数据库使用 **24张表中文枚举版**。

当前总集成负责人已经完成了项目基础框架和管理员基础资料模块，包括：

1. ASP.NET Core MVC 项目基础框架；
2. Oracle 数据库连接；
3. `campus` 用户和 24 张表中文枚举版适配；
4. 登录与角色权限控制；
5. 公共 Layout、顶部栏、侧边栏；
6. 管理员首页；
7. `nodes` 节点管理；
8. `service_types` 服务类型管理；
9. `service_node_rules` 服务节点规则绑定；
10. 上述模块均已通过页面操作和 SQL Developer 查询验证。

分工重点转向 **任务发布、接单状态流转、支付退款、评价投诉、结算审计报表、前端页面完善和联调测试**。

二、总体分工原则

本项目共 10 人，采用：

3名前端 + 5名核心后端 + 1名测试联调协助 + 1名总集成负责人

调整后的原则如下：

1. 总集成负责人已经完成公共框架和管理员基础资料模块，后续主要负责合并、联调和项目稳定性；
2. 前端成员分别负责普通用户端、跑腿员端、管理员端页面；
3. 后端成员按业务模块划分，不再按单表划分；
4. 测试联调协助成员负责测试数据、功能验收、Bug 记录和演示材料整理；
5. 每位成员都需要参与 C# 项目开发或项目联调；
6. 每位成员都要完成自己负责模块的 PPT 内容，并在答辩时讲解自己负责的部分；
7. 所有代码必须遵守现有项目结构，不允许重新创建项目；
8. 所有枚举值必须使用中文，不允许使用英文状态值。

三、当前已完成模块

总集成负责人已完成内容

已完成内容	说明
项目基础框架	ASP.NET Core MVC 项目已搭建
Oracle 连接	使用 Oracle.ManagedDataAccess.Core 连接 Oracle
数据库结构	24 张表中文枚举版已建好
登录权限	管理员登录、Session、角色识别已完成
公共页面	统一顶部栏、侧边栏、Layout 已完成
管理员首页	管理员登录后可进入后台首页
节点管理	列表、新增、详情、修改、删除已测试通过
服务类型管理	列表、新增、详情、修改、删除已测试通过
服务节点规则	绑定、列表、重复绑定提示、删除已测试通过
数据库验证	页面操作后已通过 SQL Developer 验证数据库变化

后续不再重复分配的模块

以下模块已完成，不再作为其他组员主要任务：

nodes
service_types
service_node_rules
管理员基础资料页面
项目公共登录框架
OracleDbHelper
公共 Layout

其他组员后续可以调用这些模块，但不要随意重写。

四、新版10人分工总表

组 员	角色	主要负责内容	涉及表	当前 状态
组 员 1	前端1： 普通用户端	普通用户首页、地址管理、任务发布、我的任务、支付、确认签收、评价投诉页面	users、user_addresses、tasks、payments、reviews、complaints	待 完 成

组 员	角 色	主要负责内容	涉及表	当前状态
组 员 2	前端2: 跑腿员端	跑腿员申请、任务大厅、抢单、我的接单、状态更新、结算查看页面	runners、assign_records、task_status_logs、settlements	待完成
组 员 3	前端3: 管理员端	跑腿员审核、派单重派、退款审核、投诉处理、审计日志、统计报表页面	runners、assign_records、refunds、complaints、audit_logs、reports	待完成
组 员 4	后端1: 账户、地址、跑腿员模块	用户注册登录完善、地址管理、跑腿员申请、管理员审核跑腿员	users、user_addresses、runners	待完成
组 员 5	后端2: 任务发布模块	任务主单、外卖分发、快递代取、私人任务、任务详情、我的任务	tasks、food_delivery_details、express_pickup_details、private_task_details	待完成
组 员 6	后端3: 接单、派单、状态流转模块	任务大厅、抢单、派单、重派、状态日志、确认签收	assign_records、task_status_logs、tasks、runners	待完成
组 员 7	后端4: 支付、退款、评价、投诉模块	模拟支付、退款申请审核、评价、投诉、信誉分联动	payments、refunds、reviews、complaints、runners	待完成
组 员 8	后端5: 结算、审计、报表模块	跑腿员结算、结算明细、审计日志、统计报表	settlements、settlement_payment_items、audit_logs、audit_status_log_checks、audit_payment_checks、audit_refund_checks、reports、report_audit_items	待完成
组 员 9	测试联调与文档协助	测试数据、功能验收、Bug记录、演示截图、协助后端模块联调	全部业务表	待完成

组员	角色	主要负责内容	涉及表	当前状态
组员10	总集成负责人	公共框架、Oracle连接、登录权限、Layout、Git合并、联调、演示流程；已完成管理员基础资料模块	全部模块公共部分	基础部分已完成

五、前端组详细分工

组员1：普通用户端前端

负责范围

组员1负责普通用户登录系统后的全部用户端页面，重点展示普通用户从维护地址、发布任务、支付订单到确认签收、评价投诉的完整使用流程。

主要页面

1. 普通用户首页；
2. 用户地址管理页面；
3. 新增地址页面；
4. 修改地址页面；
5. 发布任务入口页面；
6. 外卖分发任务发布页面；
7. 快递代取任务发布页面；
8. 私人任务发布页面；
9. 我的任务列表页面；
10. 任务详情页面；
11. 模拟支付页面；
12. 确认签收页面；
13. 评价页面；
14. 投诉页面。

前端要求

1. 页面风格与当前系统 Layout 保持一致；
2. 页面表单字段必须和后端 ViewModel 对齐；
3. 所有状态显示为中文；
4. 发布任务时应能选择地址、服务类型和节点；

- 5. 如果服务类型与节点不匹配，应显示友好提示；
- 6. 我的任务页面应能显示任务状态；
- 7. 支付、确认签收、评价投诉按钮应根据任务状态显示；
- 8. 页面不直接访问数据库，只调用后端功能。

需要配合的后端成员

功能	后端负责人
地址管理	组员4
发布任务	组员5
我的任务	组员5、组员6
支付	组员7
评价投诉	组员7

答辩讲解内容

- 1. 普通用户端整体页面结构；
- 2. 用户如何维护地址；
- 3. 三类任务如何发布；
- 4. 用户如何支付、确认签收、评价投诉；
- 5. 用户端与后端模块如何配合。

组员2：跑腿员端前端

负责范围

组员2负责跑腿员端页面，重点展示跑腿员从申请、查看任务大厅、抢单、执行任务、更新状态到查看结算的完整流程。

主要页面

- 1. 跑腿员申请页面；
- 2. 跑腿员首页；
- 3. 任务大厅页面；
- 4. 抢单结果提示页面；
- 5. 我的接单列表；
- 6. 接单详情页面；
- 7. 任务状态更新页面；
- 8. 完成凭证上传页面；
- 9. 我的结算记录页面；
- 10. 跑腿员个人状态页面。

前端要求

- 1. 跑腿员端页面只允许“跑腿员”角色访问；
- 2. 任务大厅要展示任务标题、服务类型、节点、价格、状态；
- 3. 抢单按钮要有明显反馈；
- 4. 任务状态更新按钮要根据当前状态变化；
- 5. 我的接单页面要展示接单时间、任务状态、配送节点等；
- 6. 我的结算页面只负责查看，不允许修改结算数据。

需要配合的后端成员

功能	后端负责人
跑腿员申请	组员4
任务大厅	组员6
抢单	组员6
状态更新	组员6
结算查看	组员8

答辩讲解内容

- 1. 跑腿员如何申请；
- 2. 跑腿员如何查看任务大厅；
- 3. 跑腿员如何抢单；
- 4. 跑腿员如何更新状态；
- 5. 跑腿员如何查看收入结算。

组员3：管理员端前端

负责范围

组员3负责管理员端剩余页面。注意：节点管理、服务类型管理、服务节点规则模块已经由总集成负责人完成，组员3不需要重写这些内容，只需要保持页面风格一致，并负责其他管理员功能页面。

主要页面

- 1. 跑腿员审核页面；
- 2. 用户管理页面；
- 3. 管理员派单页面；
- 4. 管理员重派页面；
- 5. 退款审核页面；
- 6. 投诉处理页面；
- 7. 审计日志页面；

8. 统计报表页面；
9. 订单管理页面；
10. 跑腿员绩效页面。

前端要求

1. 管理员页面只允许“管理员”角色访问；
2. 表格样式与已完成的节点管理页面保持一致；
3. 审核、处理、派单、重派按钮要清晰；
4. 删除或处理类操作要有确认提示；
5. 错误信息不要直接显示 Oracle 原始异常；
6. 报表页面可以先使用表格展示统计结果。

需要配合的后端成员

功能	后端负责人
跑腿员审核	组员4
派单重派	组员6
退款审核	组员7
投诉处理	组员7
审计日志	组员8
统计报表	组员8

答辩讲解内容

1. 管理员端整体页面；
2. 跑腿员审核页面；
3. 派单重派页面；
4. 退款投诉处理页面；
5. 审计和统计页面的管理意义。

六、后端组详细分工

组员4：账户、地址、跑腿员模块

负责表

```
users
user_addresses
runners
```

负责功能

- 1. 用户注册逻辑完善；
- 2. 用户登录逻辑补充；
- 3. 用户信息查询；
- 4. 用户信息修改；
- 5. 地址列表查询；
- 6. 新增地址；
- 7. 修改地址；
- 8. 删除地址；
- 9. 设置默认地址；
- 10. 跑腿员申请；
- 11. 管理员审核跑腿员；
- 12. 跑腿员工作状态维护；
- 13. 禁用或恢复用户账号。

关键业务逻辑

- 1. 普通用户可以申请成为跑腿员；
- 2. 跑腿员申请后默认审核状态为“待审核”；
- 3. 管理员审核通过后，跑腿员状态可变为“空闲”；
- 4. 地址表是弱实体表，主键为 `user_id + address_no`；
- 5. 同一用户可以有多个地址；
- 6. 同一用户只能有一个默认地址；
- 7. 禁用账号不能登录系统。

中文枚举要求

字段	可用值
users.user_role	普通用户、跑腿员、管理员
users.account_status	正常、禁用
user_addresses.is_default	是、否
runners.audit_status	待审核、已通过、已拒绝
runners.work_status	空闲、忙碌、离线

建议文件

```
Controllers/UserController.cs
Controllers/AddressController.cs
Controllers/RunnerController.cs

Services/UserService.cs
```

```
Services/AddressService.cs
Services/RunnerService.cs

Repositories/UserRepository.cs
Repositories/AddressRepository.cs
Repositories/RunnerRepository.cs

ViewModels/UserViewModel.cs
ViewModels/AddressViewModel.cs
ViewModels/RunnerViewModel.cs
```

验收标准

1. 普通用户可以注册;
2. 用户可以新增、修改、删除地址;
3. 默认地址设置正确;
4. 普通用户可以提交跑腿员申请;
5. 管理员可以审核跑腿员;
6. 数据库 `users`、`user_addresses`、`runners` 发生正确变化;
7. 所有状态值均为中文。

组员5：任务发布模块

负责表

```
tasks
food_delivery_details
express_pickup_details
private_task_details
```

负责功能

1. 创建任务主单;
2. 发布外卖分发任务;
3. 发布快递代取任务;
4. 发布私人任务;
5. 查询任务列表;
6. 查询任务详情;
7. 根据任务类型显示不同明细;
8. 普通用户查看自己的任务;
9. 管理员查看全部任务;
10. 任务取消前置判断;
11. 任务价格保存;
12. 任务初始状态设置。

关键业务逻辑

- 1. 所有任务必须先写入 `tasks` 主单表;
- 2. 外卖任务再写入 `food_delivery_details`;
- 3. 快递任务再写入 `express_pickup_details`;
- 4. 私人任务再写入 `private_task_details`;
- 5. `tasks.publisher_user_id` 是发布用户;
- 6. `tasks.node_id` 是交接节点;
- 7. `tasks.service_type_id + node_id` 必须存在于 `service_node_rules`;
- 8. 任务创建时要检查服务类型和节点是否合法;
- 9. 任务状态初始可设为“已创建”;
- 10. 支付后由支付模块改为“已支付”或“待接单”。

特别注意

不要写错字段名：

正确字段	错误写法
<code>publisher_user_id</code>	<code>user_id</code>
<code>node_id</code>	<code>support_node_id</code>
<code>private_task_details</code>	<code>private_item_details</code>

三类明细表都是联合主键：

<code>task_id + detail_no</code>

中文枚举要求

字段	可用值
<code>tasks.urgent_flag</code>	是、否
<code>tasks.task_status</code>	已创建、已支付、待接单、已接单、已取件、配送中、待签收、已完成、已取消、退款中

建议文件

```
Controllers/TaskController.cs

Services/TaskService.cs

Repositories/TaskRepository.cs

ViewModels/TaskViewModel.cs
ViewModels/FoodDeliveryDetailViewModel.cs
ViewModels/ExpressPickupDetailViewModel.cs
ViewModels/PrivateTaskDetailViewModel.cs
```

验收标准

- 1. 能发布外卖分发任务；
 - 2. 能发布快递代取任务；
 - 3. 能发布私人任务；
 - 4. tasks 表新增主单；
 - 5. 对应明细表新增记录；
 - 6. 服务类型与节点不匹配时禁止创建；
 - 7. 我的任务列表能显示当前用户任务；
 - 8. 任务详情能正确显示不同明细。
-

组员6：接单、派单、状态流转模块

负责表

```
assign_records
task_status_logs
tasks
runners
```

负责功能

- 1. 任务大厅；
- 2. 跑腿员抢单；
- 3. 防止重复接单；
- 4. 管理员派单；
- 5. 管理员重派；
- 6. 任务状态更新；
- 7. 状态日志记录；
- 8. 用户确认签收；
- 9. 查询状态历史；
- 10. 跑腿员工作状态变化。

关键业务逻辑

- 1. 只有状态合适的任务才能被抢单；
- 2. 同一个任务不能被两个跑腿员抢到；
- 3. 抢单成功后写入 `assign_records`；
- 4. 抢单成功后更新 `tasks.task_status`；
- 5. 每次状态变化都必须写入 `task_status_logs`；
- 6. 管理员派单与跑腿员自主接单都要记录；
- 7. 重派时要保存重派原因；
- 8. 跑腿员忙碌时不能继续抢单；
- 9. 用户确认后任务才算真正完成。

中文枚举要求

字段	可用值
<code>assign_records.operation_type</code>	自主接单、管理员派单、重新派单
<code>tasks.task_status</code>	已支付、待接单、已接单、已取件、配送中、待签收、已完成
<code>runners.work_status</code>	空闲、忙碌、离线

建议文件

```
Controllers/AssignController.cs
Controllers/StatusLogController.cs

Services/AssignService.cs
Services/StatusLogService.cs

Repositories/AssignRepository.cs
Repositories/StatusLogRepository.cs

ViewModels/AssignRecordViewModel.cs
ViewModels/TaskStatusLogViewModel.cs
```

验收标准

- 1. 跑腿员能看到可接任务；
 - 2. 抢单成功后 `assign_records` 新增记录；
 - 3. 任务状态发生变化；
 - 4. 重复抢单被阻止；
 - 5. 管理员可以派单；
 - 6. 管理员可以重派；
 - 7. 每次状态变化都能在 `task_status_logs` 查到；
 - 8. 用户确认签收后任务变为“已完成”。
-

组员7：支付、退款、评价、投诉模块

负责表

payments
refunds
reviews
complaints
runners

负责功能

1. 模拟支付；
2. 支付记录查询；
3. 退款申请；
4. 管理员退款审核；
5. 用户评价；
6. 评价列表；
7. 用户投诉；
8. 管理员处理投诉；
9. 投诉成立后扣减跑腿员信誉分；
10. 支付或退款状态维护。

关键业务逻辑

1. 支付记录通过 record_id 关联接派记录；
2. 退款通过 payment_id 关联支付记录；
3. 评价通过 record_id 关联接派记录；
4. 投诉通过 record_id 关联接派记录；
5. 一个接派记录最多一条评价；
6. 投诉成立后应影响跑腿员信誉分；
7. 退款审核后应更新退款状态；
8. 支付成功后可推动任务进入后续状态。

中文枚举要求

字段	可用值
payments.pay_method	微信、支付宝、现金
payments.pay_status	未支付、已支付、支付失败、已退款
complaints / refunds 状态	按 SQL 文件中文枚举为准

建议文件

```
Controllers/PaymentController.cs
Controllers/RefundController.cs
Controllers/ReviewController.cs
Controllers/ComplaintController.cs

Services/PaymentService.cs
Services/RefundService.cs
Services/ReviewService.cs
Services/ComplaintService.cs

Repositories/PaymentRepository.cs
Repositories/RefundRepository.cs
Repositories/ReviewRepository.cs
Repositories/ComplaintRepository.cs
```

验收标准

1. 用户可以模拟支付;
2. 支付成功后 `payments` 表新增记录;
3. 用户可以申请退款;
4. 管理员可以审核退款;
5. 用户可以评价已完成任务;
6. 用户可以投诉任务;
7. 管理员处理投诉后数据库状态变化;
8. 投诉成立时跑腿员信誉分发生变化。

组员8：结算、审计、报表模块

负责表

```
settlements
settlement_payment_items
audit_logs
audit_status_log_checks
audit_payment_checks
audit_refund_checks
reports
report_audit_items
```

负责功能

1. 跑腿员结算生成;
2. 结算记录查询;
3. 结算明细查看;
4. 防止重复结算;

5. 审计日志查询;
6. 状态日志审计;
7. 支付审计;
8. 退款审计;
9. 订单数量统计;
10. 节点业务量统计;
11. 跑腿员绩效统计;
12. 退款率、投诉率统计;
13. 报表生成记录。

关键业务逻辑

1. 只有已完成且无未处理投诉的任务才能进入结算;
2. 同一支付记录不能重复结算;
3. 结算金额应扣除平台抽成;
4. 审计日志用于记录异常检查;
5. 报表可以基于任务、支付、退款、投诉、结算等数据生成;
6. 报表页面可以先用表格展示统计结果, 不要求复杂图表。

建议文件

```
Controllers/SettlementController.cs
Controllers/AuditController.cs
Controllers/ReportController.cs

Services/SettlementService.cs
Services/AuditService.cs
Services/ReportService.cs

Repositories/SettlementRepository.cs
Repositories/AuditRepository.cs
Repositories/ReportRepository.cs

ViewModels/SettlementViewModel.cs
ViewModels/AuditViewModel.cs
ViewModels/ReportViewModel.cs
```

验收标准

1. 可以生成跑腿员结算记录;
2. 可以查看结算明细;
3. 重复结算被阻止;
4. 可以查看审计日志;
5. 可以生成或查看统计报表;
6. 能统计订单量、节点业务量、跑腿员绩效;
7. 数据库对应表发生正确变化。

组员9：测试联调与文档协助

负责范围

组员9主要负责测试、联调和文档协助，同时根据项目进度协助任务发布、支付退款、状态流转等模块。

主要任务

1. 编写测试数据；
2. 协助维护 `Database/init_test_data.sql`；
3. 测试各模块页面；
4. 记录 Bug；
5. 验证数据库是否发生变化；
6. 整理测试截图；
7. 整理演示流程截图；
8. 协助后端成员做接口联调；
9. 编写系统设计与实现文档中的测试部分；
10. 协助准备答辩演示脚本。

重点测试内容

1. 登录权限测试；
2. 普通用户发布任务测试；
3. 跑腿员抢单测试；
4. 管理员派单测试；
5. 任务状态流转测试；
6. 支付退款测试；
7. 评价投诉测试；
8. 结算报表测试；
9. 中文枚举约束测试；
10. 数据库外键约束测试。

验收标准

1. 每个核心模块至少有一组测试数据；
 2. 每个核心流程至少有截图；
 3. 每个 Bug 有记录；
 4. 每个已完成模块都经过 SQL Developer 验证；
 5. 最终答辩演示流程能顺利走完。
-

组员10：总集成负责人

当前已完成内容

- 1. ASP.NET Core MVC 项目搭建；
- 2. Oracle 数据库连接；
- 3. 24 张表中文枚举版适配；
- 4. `OracleDbHelper`；
- 5. 登录与 Session；
- 6. 角色权限；
- 7. 公共 Layout；
- 8. 管理员首页；
- 9. 节点管理；
- 10. 服务类型管理；
- 11. 服务节点规则绑定；
- 12. 数据库连接测试；
- 13. 基础资料模块测试通过。

后续负责内容

- 1. 维护项目公共结构；
- 2. 合并各成员代码；
- 3. 处理 Git 冲突；
- 4. 检查项目是否能运行；
- 5. 检查登录权限是否被破坏；
- 6. 检查数据库连接是否正常；
- 7. 检查中文枚举是否被误写成英文；
- 8. 组织联调；
- 9. 准备最终演示流程；
- 10. 收集各成员 PPT 内容；
- 11. 检查最终答辩版本。

不再单独承担的新业务模块

总集成负责人后续不再单独负责以下模块的新开发：

任务发布
跑腿员抢单
支付退款
评价投诉
结算审计报表
普通用户端全部页面
跑腿员端全部页面

这些由对应组员完成。

七、模块之间的配合关系

功能流程	前端负责人	后端负责人
登录注册	组员1	组员4
地址管理	组员1	组员4
跑腿员申请	组员2	组员4
跑腿员审核	组员3	组员4
外卖任务发布	组员1	组员5
快递任务发布	组员1	组员5
私人任务发布	组员1	组员5
我的任务	组员1	组员5、组员6
任务大厅	组员2	组员6
抢单	组员2	组员6
状态更新	组员2	组员6
用户确认签收	组员1	组员6
管理员派单	组员3	组员6
管理员重派	组员3	组员6
模拟支付	组员1	组员7
退款申请	组员1	组员7
退款审核	组员3	组员7
用户评价	组员1	组员7
用户投诉	组员1	组员7
投诉处理	组员3	组员7
结算查看	组员2、组员3	组员8
审计日志	组员3	组员8
统计报表	组员3	组员8
测试与演示截图	全体配合	组员9
合并与联调	全体配合	组员10

八、开发规范

1. 分层结构

所有后端代码尽量遵循：

```
Controller -> Service -> Repository -> OracleDbHelper -> Oracle
```

说明：

- Controller：负责接收请求和返回页面；
- Service：负责业务逻辑判断；
- Repository：负责 SQL 和数据库访问；
- OracleDbHelper：负责公共数据库连接；
- ViewModel：负责页面与后端之间的数据传递；
- View：负责页面展示。

2. 不允许的行为

1. 不允许在 Controller 中直接拼 SQL；
2. 不允许在 View 页面中访问数据库；
3. 不允许随意修改 `OracleDbHelper`；
4. 不允许随意修改 `Program.cs`；
5. 不允许随意修改 `appsettings.json` 中连接名 `OracleConnection`；
6. 不允许改动 24 表建表 SQL；
7. 不允许将中文枚举改回英文枚举；
8. 不允许直接向 `main` 分支推送未测试代码。

3. 中文枚举要求

所有业务代码中必须使用中文枚举：

类型	正确值
用户角色	普通用户、跑腿员、管理员
账号状态	正常、禁用
是否标志	是、否
审核状态	待审核、已通过、已拒绝
工作状态	空闲、忙碌、离线
节点状态	正常、关闭
服务类型状态	启用、禁用

类型	正确值
任务状态	已创建、已支付、待接单、已接单、已取件、配送中、待签收、已完成、已取消、退款中
接派类型	自主接单、管理员派单、重新派单
支付方式	微信、支付宝、现金
支付状态	未支付、已支付、支付失败、已退款

禁止使用：

USER
RUNNER
ADMIN
NORMAL
DISABLED
CREATED
PAID
WAITING
Y
N

九、Git 协作建议

1. 推荐分支

分支	负责人
main	总集成维护，稳定版本
base	总集成维护，日常整合
feature/frontend-user	组员1
feature/frontend-runner	组员2
feature/frontend-admin	组员3
feature/backend-account	组员4
feature/backend-task	组员5
feature/backend-assign-status	组员6
feature/backend-payment-afterservice	组员7
feature/backend-settlement-audit	组员8
feature/test-docs	组员9
feature/integration	组员10

2. 提交格式

建议使用中文提交信息，例如：

```
完成用户地址管理
完成快递任务发布功能
修复跑腿员抢单状态判断
完成支付记录查询
补充测试数据脚本
```

不要使用：

```
update
fix
111
改了一点
```

十、各模块完成标准

每个模块完成时，必须满足：

- 1. 页面能打开；
- 2. 功能能操作；
- 3. 数据库对应表发生正确变化；
- 4. SQL Developer 能查到变化；
- 5. 角色权限正确；
- 6. 中文枚举正确；
- 7. 页面有友好提示；
- 8. `dotnet build` 没有错误；
- 9. 提交代码前已经测试；
- 10. 有至少一张功能截图。

十一、最终演示流程

项目最终应尽量跑通以下流程：

```
管理员登录
↓
查看节点、服务类型、服务节点规则
↓
审核跑腿员
↓
普通用户登录
↓
维护地址
↓
发布外卖 / 快递 / 私人任务
↓
```



十二、答辩材料分配

内容	负责人
项目背景与需求	组员10 统筹，全体补充
数据库设计说明	组员10 统筹，后端成员补充
普通用户端演示	组员1
跑腿员端演示	组员2
管理员端演示	组员3
账户地址跑腿员模块	组员4
任务发布模块	组员5
接单状态流转模块	组员6
支付退款评价投诉模块	组员7
结算审计报表模块	组员8
测试过程与结果	组员9
系统整体架构与联调	组员10

十三、总结

本次重新分工后，总集成负责人已经完成的管理员基础资料模块不再重复分配。
后续开发重点集中在：

- 1. 用户端完整流程；
- 2. 跑腿员端完整流程；
- 3. 管理员端异常处理和报表；

4. 任务发布；
5. 接单状态流转；
6. 支付退款评价投诉；
7. 结算审计报表；
8. 测试联调与答辩演示。

总集成负责人后续主要负责 **合并、联调、稳定性检查和最终演示组织**，不再单独承担新的大业务模块开发。